

Plano de Projeto e Negócios

CLEAN AIR

Autores: Erick Reis Santos de Jesus, João Pedro Santana de Oliveira Menezes, Luan Gomes Correia, Wendell Trindade dos Santos Reis.

Orientador: Jorge Luiz dos Santos.

Curso: Técnico em Desenvolvimento de Sistemas.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

SUMÁRIO

1. INTRODUÇÃO.....	3
2. ANÁLISE DE REQUISITOS.....	5
3. PROJETO DA SOLUÇÃO.....	10
4. DIFERENCIAL E INOVAÇÃO.....	18
5. DESENVOLVIMENTO.....	19
6. TESTES E VALIDAÇÃO.....	25
7. IMPLANTAÇÃO E MANUTENÇÃO.....	29
8. MODELO DE NEGÓCIOS.....	30
9. RISCOS E DESAFIOS.....	38
10. CONCLUSÃO E PRÓXIMAS ETAPAS.....	40
REFERÊNCIAS.....	42

1. INTRODUÇÃO

1.1 Apresentação do Projeto

O Clean Air é um sistema web com arquitetura em nuvem voltado para o acompanhamento de dados de diferentes ambientes através da captação de informações emitidas por microcontroladores via protocolo MQTT. A plataforma permite que organizações e indivíduos cadastrem seus dispositivos IoT, configurem painéis de visualização personalizados e monitorem variáveis ambientais — como temperatura, umidade e pressão — em tempo real, de qualquer lugar com acesso à internet.

O Clean Air foi projetado para se destacar das demais soluções existentes em monitoramento através de uma interface minimalista e de fácil utilização, especialmente para pessoas com menor proficiência tecnológica. Enquanto alternativas como ThingsBoard e Grafana oferecem vastas funcionalidades à custa de uma curva de aprendizado elevada e documentação em inglês, o Clean Air foca no essencial: captar, exibir e exportar dados de sensores de forma simples e acessível.

Em sua stack de desenvolvimento estão presentes Node.js com Express no backend, React 18 no frontend, Neon Database (PostgreSQL 16 serverless) para armazenamento de dados, hospedagem no Render (backend) e Vercel (frontend), com comunicação em tempo real via Socket.IO e protocolo MQTT para integração com os microcontroladores.

1.2 Justificativa e Problema

Com o crescimento da Internet das Coisas (IoT) e a democratização de microcontroladores acessíveis como o ESP32, tornou-se tecnicamente viável que pequenos produtores, comerciantes e estudantes criem seus próprios sistemas de monitoramento ambiental. No entanto, as soluções disponíveis no mercado apresentam interfaces complexas, documentação predominantemente em inglês e processos de configuração longos, criando uma barreira significativa para usuários sem perfil técnico avançado.

Essa barreira é confirmada por estudos de mercado. O Instituto Atlântico aponta que uma das principais barreiras à adoção da IoT é a falta de profissionais com habilidades técnicas, já que se trata de uma tecnologia complexa que exige um conjunto diverso de competências (INSTITUTO ATLÂNTICO, 2024). No mesmo sentido, diagnósticos realizados pelo Sebrae revelaram que 85% das micro e pequenas empresas atendidas ainda estão em estágio inicial de maturidade digital, e que os maiores desafios incluem estratégia, uso de dados e capacitação de pessoas (TONET, 2024). Esse cenário se reflete nos dados: dois terços dos



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

pequenos negócios brasileiros concentram-se nos níveis mais baixos de maturidade digital (SEBRAE, 2024). Esse comportamento se reflete no público-alvo do Clean Air: muitos proprietários de pequenos negócios hesitam em adotar soluções que consideram complexas e preferem métodos tradicionais que lhes oferecem familiaridade e segurança.

Este projeto foi escolhido pela equipe por identificar uma demanda real nesse segmento: pessoas como donos de pequenos mercados, produtores rurais e estudantes de eletrônica que desejam monitorar variáveis ambientais em seus espaços, mas encontram nas soluções existentes uma curva de aprendizado incompatível com sua rotina e nível de familiaridade tecnológica.

O Clean Air propõe resolver essa dor entregando uma plataforma web gratuita (em seu plano básico), em português, com configuração rápida e interface minimalista. O sistema abstrai as complexidades do protocolo MQTT e da gestão de usuários, permitindo que o usuário final configure seu ambiente de monitoramento em poucos passos e comece a visualizar seus dados imediatamente.

O problema portanto pode ser resumido em: “Uma ausência de soluções de monitoramento para IOT voltados para o público não-técnico, ou com baixa / média afinidade com tecnologia. Que impede o uso de microcontroladores em pequenos comércios.”

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver um sistema web que permita monitorar ambientes diversos através da integração com dispositivos IoT, voltado para donos de pequenos comércios e pequenas fazendas que não possuem grande proficiência em tecnologia, com a finalidade de permitir que monitorem seus locais e equipamentos através de uma interface minimalista, simplificada e de fácil uso.

1.3.2 Objetivos Específicos

- Realizar o levantamento de requisitos junto aos stakeholders.
- Modelar o banco de dados relacional.
- Desenvolver a interface focada em usabilidade (UX/UI) seguindo as heurísticas de Nielsen.
- Implementar as funcionalidades de autenticação multi-tenancy, gerenciamento de dispositivos MQTT e dashboards em tempo real.
- Estruturar o sistema para que seja capaz de receber manutenções incrementais sem problemas.
- Abstrair as funcionalidades mais necessárias e de uso mais frequente para o sistema.

- Criar um sistema web que capte dados via protocolo MQTT, com foco em minimalismo e usabilidade.

2. ANÁLISE DE REQUISITOS

2.1 Definição do Escopo

O escopo do projeto ficou dividido entre o que o sistema fará e o que ele não fará.

Sobre o que o sistema fará:

- Realizará o monitoramento de qualquer ambiente, independente do tipo de sensor.
- Garantirá a abstração de quem poderá acessar cada espaço (multi-tenancy por domínio).
- Armazenará o histórico de dados recebidos em cada ambiente, permitindo posteriores downloads em CSV.
- Permitirá a personalização do ambiente para definir o layout dos dashboards e quais widgets estarão presentes.

Sobre o que o sistema não fará:

- Não funcionará como canal de comunicação entre gerentes e usuários.
- Não garantirá captar e exibir dados de localização (latitude e longitude).
- Não proporá dashboards com suporte ao monitoramento de sensores hospitalares.
- Não enviará alertas automáticos por e-mail, SMS ou push notification

2.2 Análise de Requisitos

2.2.1 Requisitos Funcionais (RF)

ID	Descrição do Requisito	Prioridade
RF001 — Registro de Gerente (Criação de Domínio)	O Sistema deve permitir um usuário se registrar como gerente fornecendo nome de usuário, e-mail, senha, nome do domínio e código único do domínio. O sistema cria o domínio, vincula o usuário como admin (role='admin', has_access=1) e retorna um JWT.	Alta
RF002 — Registro de Usuário Comum	O Sistema deve permitir o registro como usuário comum fornecendo nome de usuário, e-mail, senha e código de um domínio existente. Opcionalmente informa IDs de dispositivos. Cria usuário com role='user', has_access=0.	Alta
RF003 — Login de Usuário	O Sistema deve possuir autenticação com e-mail, senha e código de domínio. Após validação, retorna JWT. Exceção: superadmin não necessita de código de domínio. Sessões expiradas redirecionam ao login.	Alta



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

ID	Descrição do Requisito	Prioridade
RF004 — Recuperação de Dados do Usuário Autenticado	O Sistema deve possuir Endpoint protegido que retorna os dados do usuário autenticado (id, username, email, role, has_access, domain_id) com base no JWT.	Média
RF005 — Criação de Domínio	O sistema deve fazer com que o domínio criado automaticamente no registro do gerente. Deve Armazenar id, name, code (único), admin_id e created_at. O código é usado como chave de acesso no cadastro de novos usuários.	Alta
RF006 — Listagem de Usuários e Dispositivos do Domínio	O gerente deve poder listar usuários e dispositivos associados ao seu domínio, filtrados pelo domain_id do usuário autenticado.	Média
RF007 — Cadastro de Dispositivo	O gerente deve poder cadastrar um dispositivo informando nome, broker MQTT, porta, tópico e credenciais opcionais. Dispositivo vinculado automaticamente ao domain_id do gerente.	Alta
RF008 — Atualização e Remoção de Dispositivo	O sistema deve permitir editar dados ou remover um dispositivo. A remoção exclui em cascata widgets, dados MQTT históricos e vínculos com usuários.	Alta
RF009 — Controle de Acesso de Usuários a Dispositivos	O sistema deve possibilitar que o gerente visualize solicitações pendentes e aprove ou rejeite cada uma. A aprovação deve ser registrada (device_id, user_id) em device_users e atualiza has_access=1.	Alta
RF010 — Solicitação de Acesso pelo Usuário	O Sistema deve permitir que o usuário comum solicite acesso a dispositivos do domínio. A solicitação deve ficar 'pending' até aprovação ou rejeição pelo gerente.	Alta
RF011 — Recepção de Dados via MQTT	O Backend do sistema deve assinar automaticamente os tópicos dos dispositivos cadastrados. Validar payload JSON (valores numéricos) e persistir em mqtt_data. Payloads inválidos são registrados para diagnóstico.	Alta
RF012 — Exibição de Dashboard em Tempo Real	O sistema deve possibilitar exibir um dashboard por dispositivo com widgets, atualizando os dados em tempo real via WebSocket (Socket.IO) sem recarregar a página.	Alta
RF013 — Consulta de Histórico de Leituras	O sistema deve possibilitar consultar dados históricos de leituras, filtrando por dispositivo e período (últimas 24 horas ou semana), exibindo data, hora e valores do payload.	Alta
RF014 — Consulta de Excedências (Thresholds)	O sistema deve possibilitar consultar registros em que valores do payload ultrapassaram limites configurados em widgets, com possibilidade de enviar alertas via Telegram.	Média
RF015 — Exportação de Dados em CSV	O sistema deve possibilitar exportar dados da última semana em formato CSV com BOM UTF-8, separador ponto-e-vírgula (padrão PT-BR) e compatibilidade com Microsoft Excel.	Alta

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

ID	Descrição do Requisito	Prioridade
RF016 — Criação e Configuração de Widgets	O sistema deve possibilitar ao gerente adicionar widgets, informando nome, tipo de gráfico, configuração (campos do payload e thresholds mínimo e máximo) e posição no layout.	Alta
RF017 — Atualização e Remoção de Widgets	O sistema deve possibilitar editar a configuração e a posição de um widget ou removê-lo definitivamente.	Média
RF018 — Sistema definição de funcionalidades por planos	O sistema deve possibilitar aos mantenedores de domínio administrarem e selecionarem o plano de uso desejado, com base em seu contexto operacional.	Alta
RF019 — Aproveitamento de contas inicialmente usuárias	O sistema deve possibilitar que contas criadas inicialmente como usuários comuns possam criar seus próprios domínios ou migrar para outros domínios posteriormente.	Alta

2.2.2 Requisitos Não-Funcionais (RNF)

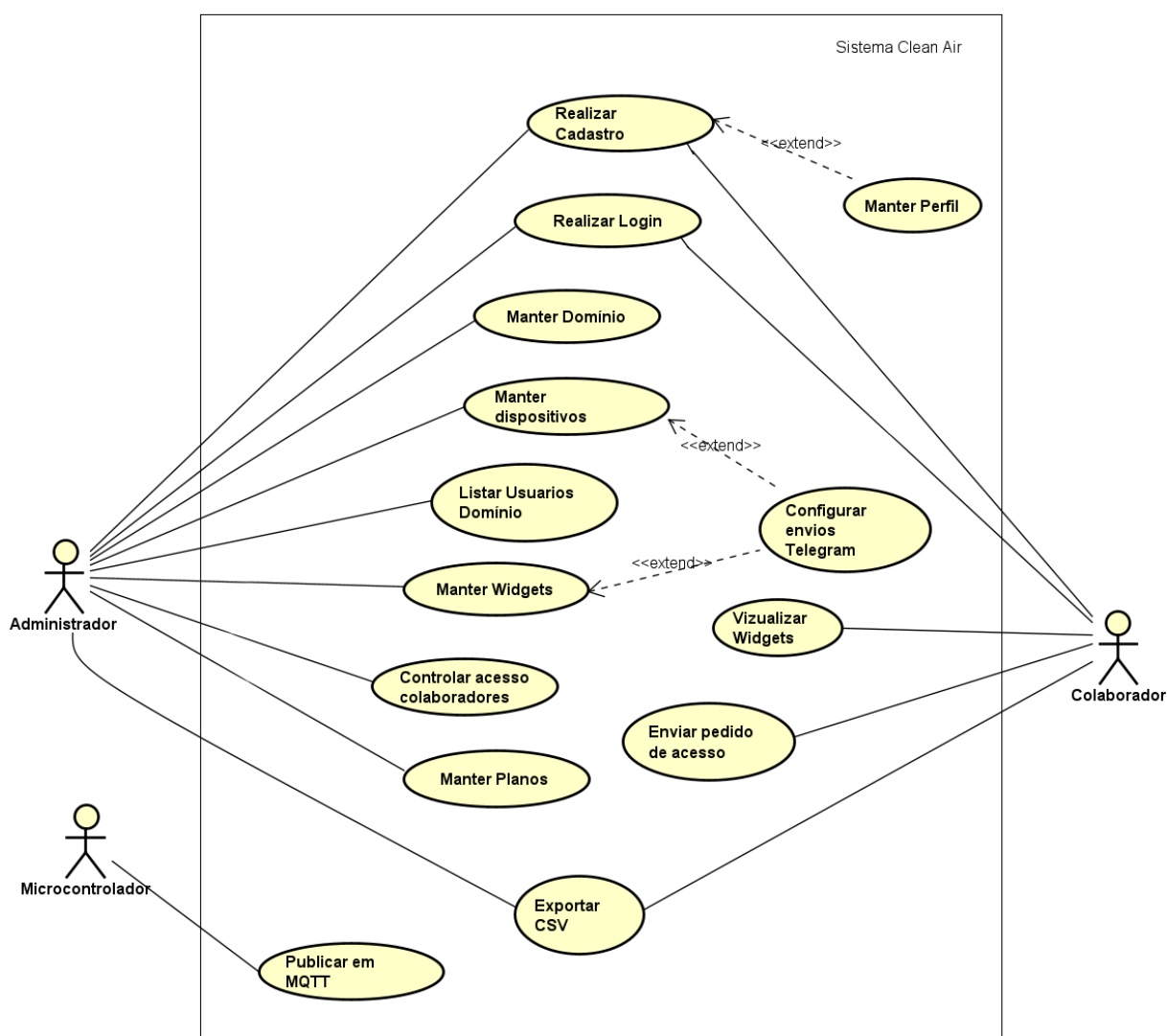
ID	Descrição do Requisito	Prioridade
RNF001	O sistema deve estar disponível 24h/7d para envio e visualização de dados dos sensores.	Alta
RNF002	O sistema deve autenticar usuários via JWT, verificando o domínio em todas as rotas protegidas, e armazenar senhas com hash bcrypt (salt rounds = 10).	Alta
RNF003	O sistema deve aplicar controle de acesso baseado em papéis (RBAC), restringindo endpoints de criação, edição e exclusão a usuários com role='admin', e permitindo a usuários comuns acessar apenas dispositivos vinculados via device_users.	Alta
RNF004	O sistema deve aplicar headers de segurança via Helmet.js, incluindo X-Content-Type-Options, X-Frame-Options e Content-Security-Policy.	Alta
RNF005	O sistema deve processar e armazenar leituras MQTT com latência inferior a 2 segundos entre a publicação no broker e a exibição no dashboard.	Alta
RNF006	O sistema deve suportar múltiplos domínios, dispositivos e usuários simultâneos sem degradação de desempenho, utilizando multi-tenancy por domain_id.	Média
RNF007	O sistema deve disponibilizar interface web acessível por navegadores modernos em dispositivos desktop e mobile, com design responsivo.	Média
RNF008	O sistema deve validar todos os payloads via Zod antes da persistência, rejeitando e registrando para diagnóstico valores não numéricos.	Alta
RNF009	O sistema deve organizar o código de forma modular, com separação em camadas (routes, controllers, models, services, middleware, schemas) e frontend componentizado em React.	Média

SENAI REGIONAL METROPOLITANA Lauro de Freitas/Bahia

ID	Descrição do Requisito	Prioridade
RNF010	O sistema deve armazenar timestamps com timezone (TIMESTAMP WITH TIME ZONE) e exibi-los no fuso horário de Brasília (America/Sao_Paulo).	Alta
RNF011	O sistema deve disponibilizar documentação formal da API REST no padrão Swagger/OpenAPI.	Média
RNF012	O sistema deve possuir paginação em seus registros de dados recebidos para melhor organização e desempenho do sistema	Média

2.2.3 Casos de Uso

O diagrama de casos de uso a seguir consolida as interações entre os atores do sistema (Administrador, Colaborador e Microcontrolador) e as funcionalidades do Clean Air.





SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

Figura 1 — Diagrama de Casos de Uso do Clean Air.

2.3 Estudo de Viabilidade

Viabilidade Técnica: A equipe possui o conhecimento técnico necessário nas tecnologias utilizadas (Node.js, React, PostgreSQL, protocolo MQTT), adquirido ao longo do Curso Técnico em Desenvolvimento de Sistemas. As ferramentas escolhidas possuem ampla documentação e comunidade ativa, reduzindo os riscos de implementação. O uso de serviços gerenciados em nuvem (Neon, Render, Vercel) elimina a necessidade de administração de infraestrutura própria.

Viabilidade Econômica: Para o ambiente de desenvolvimento e demonstração do TCC, todos os serviços utilizados oferecem planos gratuitos que cobrem o escopo do projeto sem custo. Para a versão comercial, o custo fixo de infraestrutura é de R\$ 644,00/mês (Vercel, Render e Neon); a receita do plano Comercial (R\$ 35,00/mês) a partir de aproximadamente 19 assinantes já cobre esse custo de infraestrutura. A análise financeira completa — investimento, custos operacionais, projeções e indicadores de retorno — está detalhada na Seção 8.6.

2.4 Personas e Usuários-Alvo

As personas a seguir consolidam os perfis identificados nas pesquisas de produto e de mercado, representando os principais segmentos atendidos pelo Clean Air:

- **João, 33 anos** — dono de um mercado de bairro, com pouco domínio de tecnologia. Precisa monitorar temperatura e umidade de equipamentos de refrigeração, como câmaras frias e refrigeradores, evitando perdas por falhas.
- **Ana Ribeiro, 35 anos** — produtora rural com estufa de tomate-cereja em Feira de Santana (BA). Deseja monitorar temperatura e umidade da estufa remotamente sem sair do campo. Valoriza interface em português e suporte ágil.
- **Renato Oliveira, 17 anos** — estudante do ensino médio que estuda microcontroladores e sensores. Usa o Clean Air para visualizar dados de projetos pessoais com ESP32.
- **Carlos Ferreira, 27 anos** — técnico autônomo em manutenção de sistemas de ar-condicionado. Usa microcontroladores em projetos freelance e quer oferecer monitoramento remoto dos equipamentos instalados sem configurar infraestrutura de servidor própria.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

3. PROJETO DA SOLUÇÃO

3.1 Arquitetura de Software

O padrão arquitetural utilizado pelo Clean Air é o MVC (Model-View-Controller), amplamente adotado em projetos web por sua clara separação de responsabilidades: o Model gerencia os dados e a lógica de negócio (interação com o banco PostgreSQL via driver pg), a View representa a camada de apresentação (frontend em React 18) e o Controller processa as requisições HTTP recebidas pelo Express e coordena as respostas entre as demais camadas.

Essa separação facilita a manutenção do código, permite que membros da equipe trabalhem em camadas distintas de forma independente e torna a aplicação mais testável e escalável. A camada de serviços (services/) complementa o MVC centralizando lógicas transversais como a gestão de conexões MQTT e a emissão de eventos via Socket.IO, que não se encaixam diretamente em nenhuma das três camadas clássicas.

3.2 Modelo de Dados

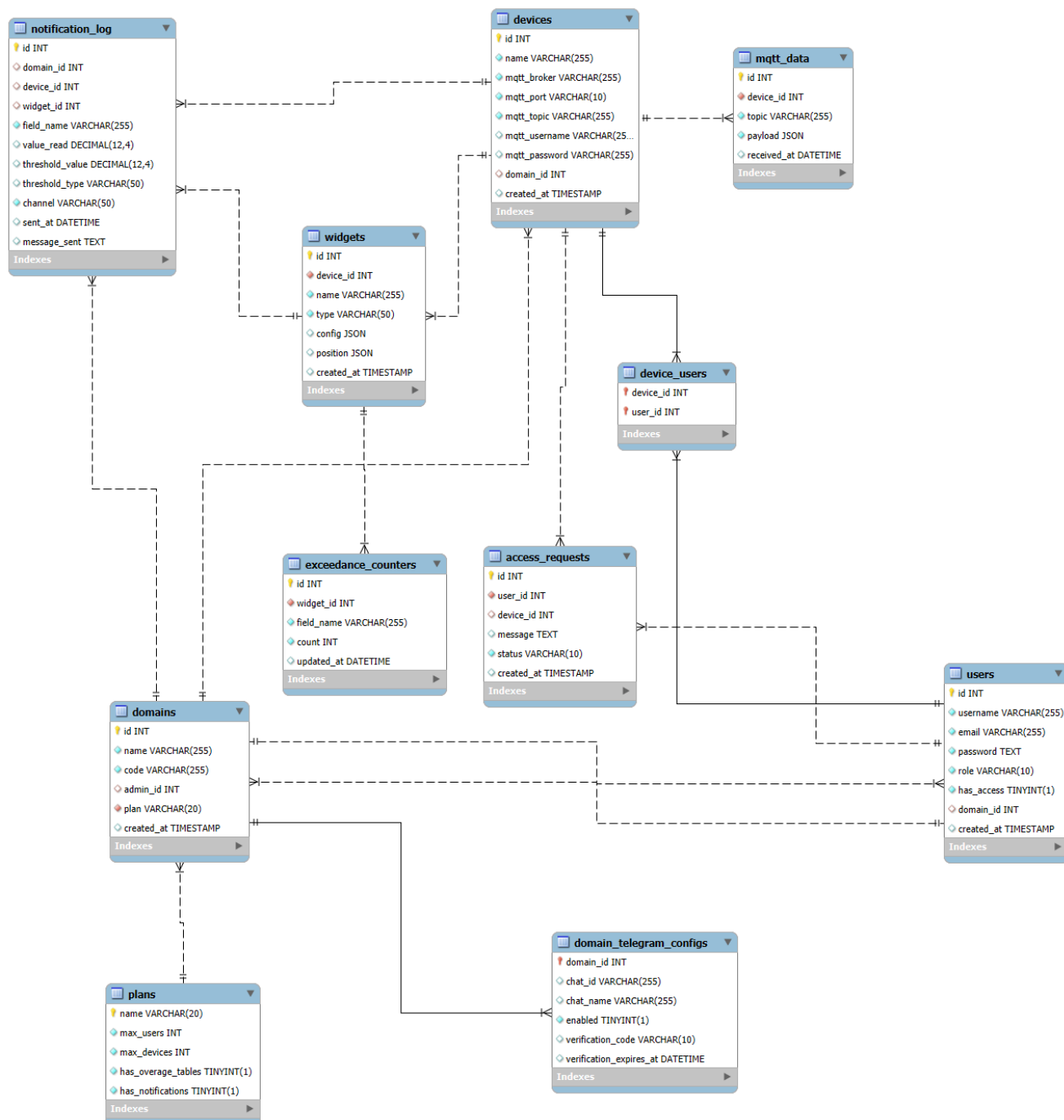


Figura 2 — Diagrama Entidade-Relacionamento.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

- **access_requests :**

Tabela em que são armazenados as solicitações de acesso, para visualização de determinado microcontrolador, sendo possível ver quem solicitou acesso, assim como o status que se encontra a mesma.

- **device_users:**

Tabela Associativa que armazena as relações entre os dispositivos / microcontroladores com o usuário que está associado a esse.

- **devices:**

Tabela que armazena dados sobre cada microcontrolador ESP-32, informando seu nome, qual broker mqtt está ligado, sua porta, a que tópico pertence, user e senha do protocolo mqtt como por fim, o domínio a que está ligado.

- **mqtt_data:**

Tabela que armazena dados enviados pelos microcontroladores (dispositivos) físicos, relacionando o id, o dado estrangeiro do dispositivo, a mensagem que foi enviada em payload, como também os dados de data e hora.

- **users:**

Tabela responsável por armazenar os dados de todos que são usuários, sejam eles usuários ou administradores. Nela estão contidos os dados referentes ao nome de usuário, email, senha, o identificador de usuário ou administrador, se já acessou o sistema alguma vez, como por fim o identificador de domínio e a data em que foi criada.

- **domains:**

Tabela que torna possível a arquitetura *Multi-Tenant* (arquitetura que permite que múltiplos usuários utilizem a mesma instância de uma solução, porém com acessos, dados e permissões isolados) operar, servindo como forma de agrupar os usuários e dispositivos em seus respectivos blocos. Nela estão contidos dados como: seu nome de domínio, seu código, o código do administrador (aquele que criou), o plano ao qual pertence, como por fim, a data em que foi criada.

- **widgets:**

Tabela responsável por armazenar os dados ligados aos widgets (dashboards) de visualização. Cada widget está vinculado a um dispositivo e possui um nome, um tipo (ex: gráfico de linha, pizza), uma configuração em JSON que define seus parâmetros visuais e de limite, e uma posição no layout, também em JSON, que permite o posicionamento livre na tela.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

- **exceedance_counters:**

Tabela associativa que mantém contadores de quantas vezes um determinado campo de um widget ultrapassou um limite configurado. Está ligada diretamente a um widget e registra o nome do campo monitorado, o total de ocorrências (count). Serve como base para regras de notificação com controle de frequência.

- **notification_log:**

Tabela de histórico de notificações disparadas pelo sistema. Registra cada evento em que um valor lido, ultrapassou um limite configurado, indicando o tipo de limite (mínimo ou máximo), o canal utilizado para envio (Telegram), a mensagem que foi enviado assim como o momento exato do disparo (sent_at). Está relacionada ao domínio, dispositivo e widget que originou o alerta, permitindo rastreabilidade completa dos eventos críticos.

- **plans:**

Tabela que armazena os planos disponíveis no sistema, sendo possível ver quais limites e funcionalidades cada plano oferece a um domínio. Nela estão contidos dados como: o nome do plano, o número máximo de usuários permitidos, o número máximo de dispositivos, se permite criar a tabela de excedências, como por fim se possui acesso ao módulo de notificações.

- **domain_telegram_configs:**

Tabela que armazena os dados de conexão com o bot do Telegram de cada domínio, sendo possível ver a qual grupo ou chat o domínio está vinculado, assim como o estado dessa integração. Nela estão contidos dados como: o chave estrangeira do domínio, o id e o nome do chat conectado, se as notificações estão ativas, o código de verificação temporário gerado para realizar a conexão, como por fim a data de expiração desse código.

3.3 Diagrama de Classes UML

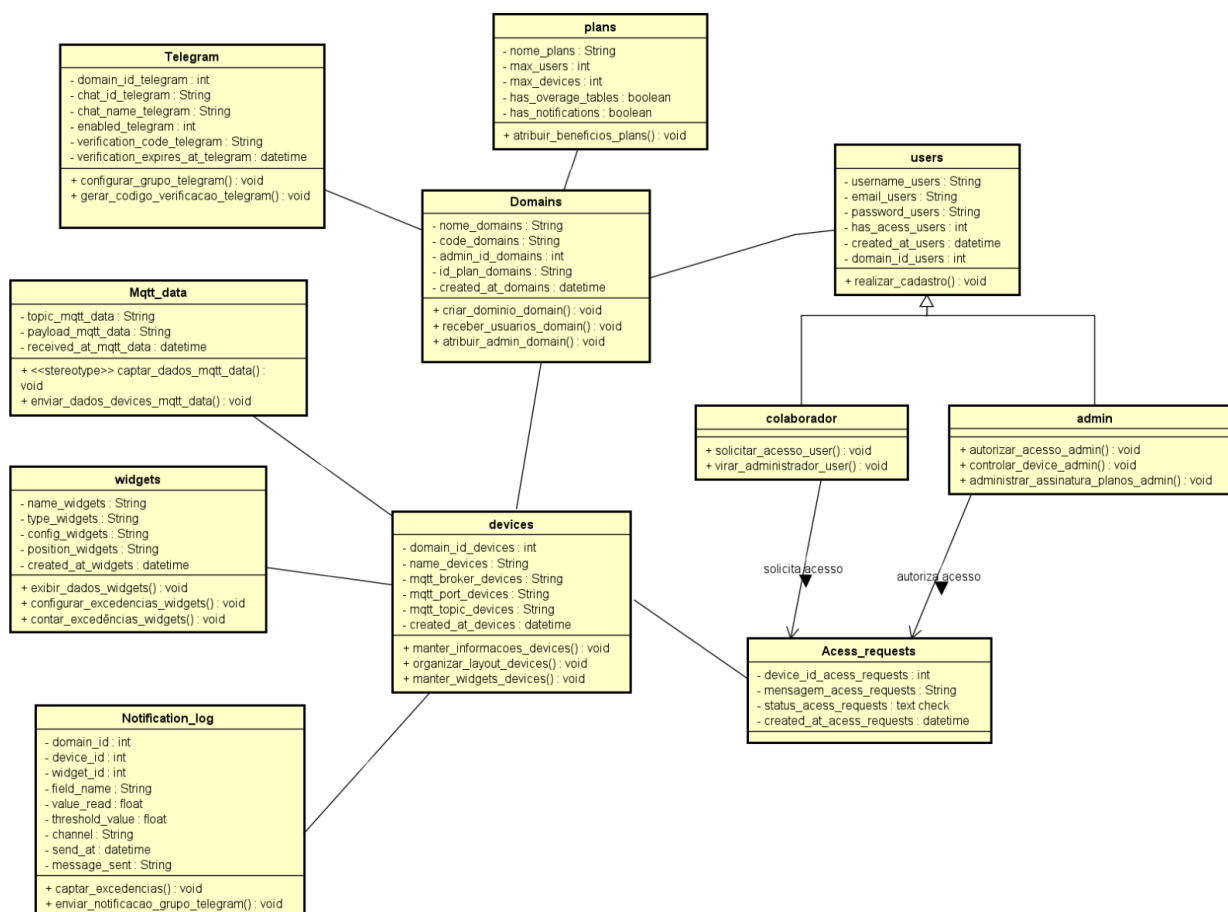


Figura 3 — Diagrama de Classes UML (inserir imagem original do repositório).

3.4 Interface com o Usuário

Nossa interface foi desenvolvida - desde seus wireframes de baixa fidelidade - para ser intuitiva ao nosso público-alvo, para atingirmos esse objetivo, recorremos a algumas heurísticas de Nielsen, tais como:

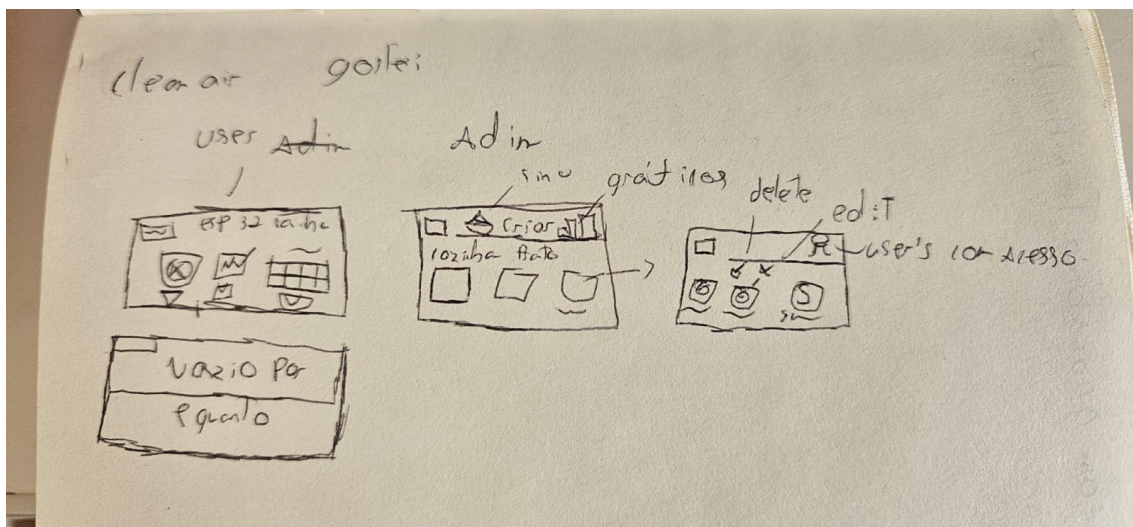
- **Estética e desing minimalista:** Os protótipos abaixo deixam bem claro o seguimento dessa heurística, buscamos reduzir ao máximo a presença de várias opções presentes na tela ao mesmo tempo, trazendo esse cuidado de não desviar a atenção dos usuários ao que é realmente relevante;
- **Reconhecimento ao invés de memória e Correspondência entre sistema e o mundo real:** Em nosso sistema foi planejada a utilização de ícones já conhecidos, por estarem presentes em vários outros sistemas, principalmente redes sociais, buscamos, portanto auxiliar a utilização dos nossos usuários, os poupando da necessidade de terem de se adaptar a ícones novos. Os

SENAI REGIONAL METROPOLITANA Lauro de Freitas/Bahia

exemplos de ícones presentes que se enquadram nessas características: O ícone de perfil, o ícone de casa, o ícone do sino, o ícone do lápis, da lixeira e o ícone do excel;

- **Visibilidade de Status do sistema:** O protótipo deixa isso claro com a mensagem de “Esperando receber acesso ao(s) dashboards”, demonstrando o cuidado demonstrado desde a idealização de garantir um feedback aos usuários, informando-os sobre o status em que se encontram
- **Consistência e padrões:** Com o cumprimento dessa heurística desde os protótipos, fica demonstrado o cuidado em que foi pensado para que todo o sistema do Clean Air seguisse um padrão de linguagem, cores, fontes e formas, garantindo portanto uma boa experiência que permitisse a consolidação do aprendizado do usuário com a solução.

Portanto, como descrito acima, o nosso planejamento para que nossas interfaces pudessem desde sua concepção serem intuitivas partiram do cumprimento de algumas heurísticas propostas por Nielsen





SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia



Esperando receber acesso ao(s) dashboard(s)...



Figura 4 — Wireframes e protótipos de alta fidelidade.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

3.5 Tecnologias Escolhidas

Camada	Tecnologia
Frontend	React.js 18 + Chart.js + Socket.IO Client
Backend	Node.js + Express 4 + Socket.IO Server
Banco de Dados	PostgreSQL 16 — Neon (serverless cloud)
Autenticação	JWT (jsonwebtoken) + bcryptjs
Validação	Zod 3
Segurança HTTP	Helmet 7
Protocolo IoT	MQTT 5 (biblioteca mqtt)
Infraestrutura	Vercel (frontend) / Render (backend)
Hospedagem do Repositório	Github
CI/CD	Github Actions
Documentação Api	Swagger
Envio de exceções	Telegram

4. DIFERENCIAL E INOVAÇÃO

4.1 Análise de Concorrentes

A análise a seguir compara o Clean Air com os principais concorrentes diretos em monitoramento IoT:

Concorrente	Dificuldade de Uso	Dashboards?	Tempo de Config. para Uso	Gratuito?	Português?
Thingsboard	Complexo	Sim	Longo	Sim, limitado	Não
Blynk	Mediano	Sim	Rápido	Sim, muito limitado	Não
Grafana	Complexo	Sim	Longo	Sim, limitado	Não
Clean Air	Simples	Sim	Rápido	Sim, limitado	Sim

4.2 Diferenciais Inovadores

O diferencial do Clean Air vem da simplicidade. Assim como os concorrentes, ele capta dados enviados via protocolo MQTT por microcontroladores IoT; porém, apresenta-se em uma interface acessível, de poucos passos para configuração e sem uma vastidão de funcionalidades, concentrando-se nos recursos que servem à grande maioria do público-alvo (gráficos de linha, barra e gauge, com thresholds configuráveis).

Além da interface, o Clean Air é a única solução comparada inteiramente traduzida para o português, eliminando a barreira do idioma para o público-alvo identificado.

A configuração completa — do cadastro à visualização do primeiro dado — pode ser realizada em menos de 10 minutos, enquanto concorrentes como o ThingsBoard e Grafana podem exigir um maior tempo de configuração inicial e, em vários casos, o download de recursos externos ou o uso de outros softwares em conjunto.

5. DESENVOLVIMENTO

5.1 Plano de Desenvolvimento e Cronograma

A metodologia aplicada durante o desenvolvimento foi a metodologia ágil Kanban, conduzida pela ferramenta Trello para acompanhamento e atribuição de tarefas. A atribuição ficava a cargo individual do time, podendo cada integrante assumir as atividades livres no quadro. Mais tardiamente, a equipe passou a organizar Sprints — ciclos de trabalho focados em partes específicas do projeto por um intervalo determinado — voltados aos pontos mais críticos para a conclusão.

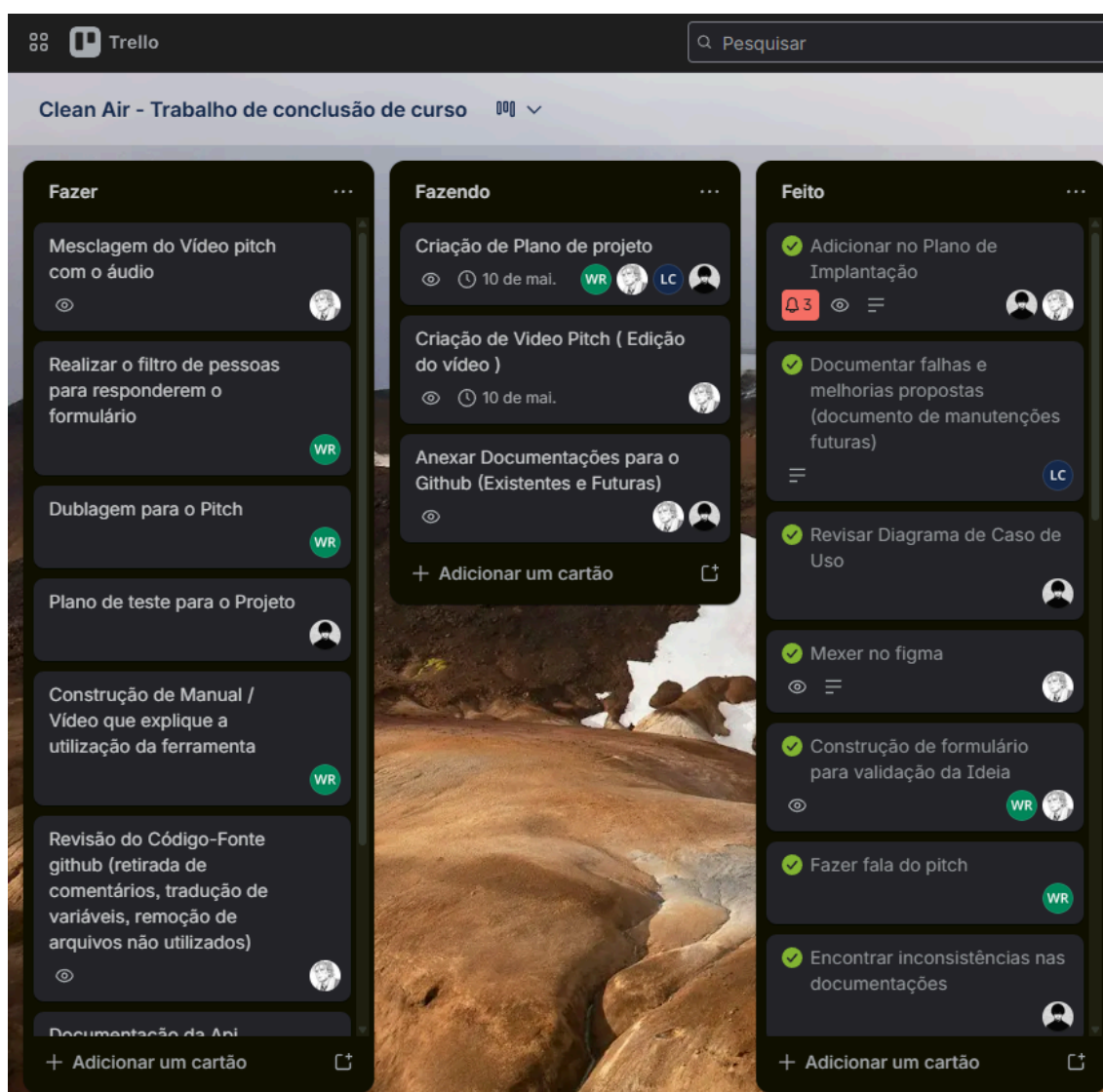


Figura 5 — Quadro Trello do projeto (inserir imagem original do repositório).



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

O cronograma geral a seguir consolida as fases do projeto e as Sprints de conclusão (evitando duplicidade entre o planejamento macro e o detalhamento por Sprint):

Fase / Sprint	Período	Atividade Principal	Entregável / Marco
Sprint 1 — Documentação	25/04 – 05/05/2026	Atualização de todas as documentações técnicas e do TCC	Documentação Atualizada e validada
Sprint 2 — Manutenção	06/05 – 20/05/2026	Correção de bugs identificados e ajustes em bibliotecas	Sistema estável e corrigido
Sprint 3 — Funcionalidades	21/05 – 05/06/2026	Implementação de funcionalidades não críticas do backlog	Funcionalidades Implementadas e validadas
Sprint 4 — Testes e Ajustes	06/06 – 12/06/2026	Testes de integração, usabilidade e desempenho; correção de defeitos	Testes aprovados
Sprint 5 — Go-Live	13/06 – 25/06/2026	Homologação final, geração de relatórios e encerramento	GO-LIVE — Entrega final

5.2 Padrões de Código e Qualidade

A equipe definiu os seguintes padrões para garantir consistência e legibilidade ao longo do projeto:

Nomenclatura:

- camelCase para variáveis, funções e propriedades JavaScript/Node.js (ex.: mqttBroker, deviceId).
- PascalCase para componentes React (ex.: DeviceCard, WidgetChart).
- snake_case para colunas e tabelas do PostgreSQL (ex.: domain_id, mqtt_topic).
- SCREAMING_SNAKE_CASE para variáveis de ambiente (ex.: JWT_SECRET, DATABASE_URL).

5.3 Ferramentas e Ambientes

O uso de Git e GitHub para versionamento está presente desde o início do projeto. A partir do semestre 2026.1, a equipe adotou o padrão de Conventional Commits para identificar o tipo de manutenção realizada em cada commit:

- **fix** — correção de um problema encontrado no sistema.
- **feat (feature)** — Na tradução, “recurso” é uma abreviação que permite identificar que nesse commit foi adicionado no sistema uma nova funcionalidade e ou recurso

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

- **docs (documentos)** — Identificação para que informar que nesse commit, foi adicionado ao repositório do sistemas documentações que permitam entender do que se trata a aplicação, os processos passados e registrados
- **refactor (refatoração)** — Identificação do commit para registrar que o sistema passou por uma reestruturação de seu código fonte, tornando-o mais legível, removendo partes não mais utilizadas como por fim, correções de vulnerabilidades.

Já a estratégia de branches adotada pela a equipe foi a *Feature Branching*, nessa abordagem, cada nova feature proposta é desenvolvida em uma branch à parte, separada da *main*, permitindo trabalhar em paralelo em novas funcionalidades sem afetar o código principal, o mantendo sempre pronto para entrar em produção, entretanto no nosso contexto o diferencial é que cada nova funcionalidade é anexada a uma única branch, a mantendo com um tempo de vida mais duradouro que o padrão da estratégia.

Por fim, no processo de desenvolvimento utilizamos o ambiente do LocalHost (quando executamos o sistema localmente, com a máquina agindo como um servidor virtual interno), já para o ambiente de testes, o Render.

5.4 Funcionalidades Implementadas

Todas as funcionalidades a seguir estão presentes e operantes em produção (cleanairsenailauro.vercel.app). Detalham-se abaixo as implementações de maior complexidade técnica, com trechos do código-fonte.

Trecho 1 — Middleware de Autenticação JWT e Controle de Acesso por Papel (RBAC). O middleware `authMiddleware.js` é chamado em todas as rotas protegidas: verifica o token JWT do cabeçalho `Authorization`, decodifica o payload e injeta os dados do usuário em `req`. O middleware `requireAdmin` verifica se o usuário possui `role='admin'`, bloqueando com 403 caso contrário.

```
// middleware/authMiddleware.js
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer <token>
  if (!token) {
    return res.status(401).json({ error: 'Token não fornecido' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, email, role, domain_id, has_access }
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token inválido ou expirado' });
  }
};
```



SENAI REGIONAL METROPOLITANA Lauro de Freitas/Bahia

```
const requireAdmin = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Acesso negado: apenas administradores' });
  }
  next();
};
module.exports = { authMiddleware, requireAdmin };
```

Uso nas rotas (devices.routes.js): leitura exige apenas authMiddleware; criação, edição e remoção exigem ambos os middlewares em sequência.

```
router.get('/', authMiddleware, deviceController.listDevices);
router.post('/', authMiddleware, requireAdmin, deviceController.createDevice);
router.put('/:id', authMiddleware, requireAdmin, deviceController.updateDevice);
router.delete('/:id', authMiddleware, requireAdmin, deviceController.deleteDevice);
```

Trecho 2 — Pipeline IoT em Tempo Real (services/mqttService.js). É o trecho de maior complexidade. A função connectDeviceMQTT é chamada para cada dispositivo cadastrado: abre uma conexão individual com o broker, assina o tópico e, ao receber uma mensagem, executa a cadeia completa: validação com Zod → persistência no PostgreSQL → emissão do evento via Socket.IO para o room do dispositivo.

```
// services/mqttService.js (trecho)
const mqtt = require('mqtt');
const { pool } = require('../database/database');
const { mqttPayloadSchema } = require('../schemas/mqttSchema');
const activeClients = new Map(); // uma conexão por dispositivo

const connectDeviceMQTT = (device, io) => {
  const { id, mqtt_broker, mqtt_port, mqtt_topic,
    mqtt_username, mqtt_password } = device;
  const client = mqtt.connect(mqtt_broker, {
    port: parseInt(mqtt_port) || 1883,
    username: mqtt_username || undefined,
    password: mqtt_password || undefined,
    clientId: `cleanair_device_${id}_${Date.now()}`,
    reconnectPeriod: 5000,
  });
  client.on('connect', () => client.subscribe(mqtt_topic));
  client.on('message', async (topic, message) => {
    let parsed;
    try { parsed = JSON.parse(message.toString()); } catch { return; }
    const result = mqttPayloadSchema.safeParse(parsed);
    if (!result.success) return; // payload rejeitado e registrado
    await pool.query(
      `INSERT INTO mqtt_data (device_id, topic, payload) VALUES ($1,$2,$3)`,
      [id, topic, message.toString()]);
    io.to(`device:${id}`).emit('mqtt_data', {
      deviceId: id, topic, payload: result.data,
      receivedAt: new Date().toISOString() });
  });
  activeClients.set(id, client);
  return client;
};
module.exports = { connectDeviceMQTT };
```

Trecho 3 — Schema de Validação Zod (schemas/mqttSchema.js). Garante que o payload seja um objeto JSON em que todos os valores são numéricos. O uso de z.record(z.string(), z.number()) aceita qualquer conjunto de chaves (temperature, humidity, pressure etc.) sem declarar cada campo, tornando o sistema agnóstico ao tipo de sensor — um dos pilares do escopo.

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

```
// schemas/mqttSchema.js
const { z } = require('zod');
// Válido: { temperature: 25.3, humidity: 61.2, pressure: 1013 }
// Inválido: { temperature: 'quente', humidity: 61.2 }
const mqttPayloadSchema = z.record(z.string(), z.number());
module.exports = { mqttPayloadSchema };
```

Trecho 4 — Inicialização Automática do Banco (database/database.js). A função `createTables` usa `CREATE TABLE IF NOT EXISTS` para criar as tabelas na primeira inicialização e `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` para migrações seguras, eliminando a execução manual de scripts SQL no Neon Console.

```
// database/database.js (trecho)
const { Pool } = require('pg');
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false } });
pool.on('connect', (client) =>
  client.query("SET timezone = 'America/Sao_Paulo'"));

const createTables = async () => {
  await pool.query(`CREATE TABLE IF NOT EXISTS domains (
    id SERIAL PRIMARY KEY, name TEXT NOT NULL,
    code TEXT UNIQUE NOT NULL, admin_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
  await pool.query(`ALTER TABLE mqtt_data
    ADD COLUMN IF NOT EXISTS received_at
    TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP`);
};
module.exports = { pool, createTables };
```

Trecho 5 — Atualização em Tempo Real no Frontend (React + Socket.IO). O `useEffect` gerencia a conexão com o Socket.IO: o componente entra na room do dispositivo ao montar, ouve o evento `'mqtt_data'` e atualiza o estado local — fazendo os gráficos `Chart.js` se redesenharem sem recarga de página. O `cleanup` garante a saída da room ao desmontar, evitando vazamentos de memória.

```
// components/Dashboard/DeviceDashboard.jsx (trecho)
import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';
const socket = io(process.env.REACT_APP_API_URL, { autoConnect: false });

export default function DeviceDashboard({ deviceId }) {
  const [liveData, setLiveData] = useState(null);
  useEffect(() => {
    socket.connect();
    socket.emit('join_device', deviceId);
    socket.on('mqtt_data', (data) => {
      if (data.deviceId === deviceId) setLiveData(data);
    });
    return () => {
      socket.emit('leave_device', deviceId);
      socket.off('mqtt_data');
      socket.disconnect();
    };
  }, [deviceId]);
  return ( /* widgets recebem liveData como prop */ );
}
```



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

Demais funcionalidades operantes em produção:

- Cadastro, edição e remoção de dispositivos IoT com configuração completa de broker, porta, tópico e credenciais MQTT.
- Solicitação e aprovação de acesso de usuários a dispositivos dentro do domínio (tabelas access_requests + device_users).
- Criação e configuração de widgets — gráficos de linha, barra, pizza, com excedências mínimas e máximas por campo.
- Consulta de histórico de leituras filtrada por período (últimas 24h ou semana) e consulta de excedências.
- Exportação de dados históricos em CSV com BOM UTF-8, separador ponto-e-vírgula e compatibilidade com Microsoft Excel.
- Envio de Excedências via Telegram.
- Escolha de planos e definição de benefícios

6. TESTES E VALIDAÇÃO

6.1 Estratégia de Testes

Durante o desenvolvimento do Clean Air, os tipos de testes realizados foram:

- **Testes Unitários:** Testes voltados para garantir que pequenas partes isoladas dos sistema estejam funcionando corretamente e de forma independente
- **Testes de Integração:** Testes realizados para verificar se os diferentes módulos do Clean air (Banco de dados, Api, Frontend, WebSocket) estavam funcionando corretamente no conjunto
- **Testes de Sistema:** Testes realizados para garantir a compatibilidade entre os componentes do Clean Air, simulando um ambiente real de aplicação, nessa etapa o sistema foi testado de ponta a ponta, verificando o cumprimento dos requisitos funcionais, não funcionais, regras de negócio, integridade dos dados
- **Testes de Aceitação:** Testes que foram realizados por membros da equipe de desenvolvimento, assumindo papéis de usuários finais para validar se os possíveis cenários que a ferramenta seria empregada poderiam ser supridos pelo Clean Air
- **Testes de Regressão:** Testes que foram realizados após grandes implementações, nesses testes revisitou-se as funcionalidades antigas, verificando se se mantinham íntegras e operantes.

Todos os testes foram realizados de forma manual, com apoio das ferramentas Chrome DevTools, MQTT Explorer (simulação de envio de payloads), Render Logs e Neon Console.

6.2 Testes Realizados

ID	Módulo	Cenário	Entrada	Saída Esperada	Resultado
CT-001	Autenticação	Cadastro de gerente com dados válidos	isManager=true, username, email, senha (>=6 chars), domainName, domainCode únicos	201 Created: JWT, domínio criado, admin com has_access=1	PASSOU
CT-002	Autenticação	Cadastro com e-mail já existente	Email já registrado no banco	400: 'E-mail já cadastrado'	PASSOU
CT-003	Autenticação	Cadastro com código de domínio duplicado	isManager=true, domainCode já existente	400: 'Código de domínio já utilizado'	PASSOU

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

ID	Módulo	Cenário	Entrada	Saída Esperada	Resultado
CT-004	Autenticação	Cadastro de usuário comum com domainCode válido	isManager=false, domainCode de domínio existente	201: role=user, has_access=0	PASSOU
CT-005	Autenticação	Cadastro com domainCode inexistente	isManager=false, domainCode='CODEI NEXISTENTE'	404: 'Domínio não encontrado'	PASSOU
CT-006	Autenticação	Login com credenciais corretas	email, senha, domainCode corretos	200: JWT válido + dados do usuário	PASSOU
CT-007	Autenticação	Login com senha incorreta	email correto, senha errada, domainCode	401 Unauthorized	PASSOU
CT-008	Autenticação	Acesso a rota protegida sem JWT	Requisição sem header Authorization	401: 'Token não fornecido'	PASSOU
CT-009	Autenticação	Acesso a rota admin por usuário comum	JWT válido de role=user em POST /api/devices	403 Forbidden	PASSOU
CT-010	Dispositivos	Criação de dispositivo com dados válidos	name, mqttBroker, mqttPort, mqttTopic + JWT admin	201: dispositivo com domain_id do admin	PASSOU
CT-011	Dispositivos	Criação sem mqttTopic obrigatório	Body sem mqttTopic, JWT admin	400: 'Tópico MQTT é obrigatório'	PASSOU
CT-012	Dispositivos	Listagem por admin retorna todos do domínio	GET /api/devices, JWT admin	200 OK: array com todos dispositivos do domain_id	PASSOU
CT-013	Dispositivos	Listagem por user retorna apenas seus dispositivos	GET /api/devices, JWT user com 2 dispositivos vinculados	200 OK: apenas os 2 dispositivos vinculados na device_users	PASSOU
CT-014	Dispositivos	Exclusão remove cascata de widgets e mqtt_data	DELETE /api/devices/:id, JWT admin	200: dispositivo, widgets e mqtt_data removidos	PASSOU
CT-015	MQTT	Recepção de payload válido	ESP32 publica {"temperature":25.3,"humidity":61.2}	Registro em mqtt_data; evento Socket.IO emitido	PASSOU
CT-016	MQTT	Recepção de payload inválido (string)	ESP32 publica {"temperature":"quent e"}	Payload rejeitado; não inserido	PASSOU

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

ID	Módulo	Cenário	Entrada	Saída Esperada	Resultado
CT-017	MQTT	Consulta de histórico (24h)	GET /api/mqtt/:id/data?period=day, JWT válido	200 OK: array de registros das últimas 24h com data e hora em America/Sao_Paulo	PASSOU
CT-018	MQTT	Consulta de excedências com threshold	GET /api/mqtt/:id/exceedances?temperatureMin=15&temperatureMax=30	200 OK: apenas registros onde temperature < 15 ou > 30	PASSOU
CT-019	MQTT	Atualização em tempo real via WebSocket	Frontend subscrito em device:{id}; ESP32 publica novo dado	Evento < 2s; gráfico atualizado sem reload	PASSOU
CT-020	Widgets	Criação de widget com tipo válido	POST /api/widgets: name, type='line', deviceId, config={min:0,max:100}, JWT admin	201 Created: widget criado e vinculado ao deviceId	PASSOU
CT-021	Widgets	Listagem de widgets de um dispositivo	GET /api/widgets/device/:id, JWT válido	200 OK: array de widgets do dispositivo	PASSOU
CT-022	Widgets	Remoção de widget	DELETE /api/widgets/:id, JWT admin	200 OK: widget removido	PASSOU
CT-023	Controle de Acesso	Solicitação de acesso criada pelo usuário	POST /api/access: deviceId, JWT user	201 Created: solicitação com status='pending'	PASSOU
CT-024	Controle de Acesso	Admin aprova solicitação	PUT /api/access/:id/approve, JWT admin	status='approved'; device_users criado; has_access=1	PASSOU
CT-025	Controle de Acesso	Admin rejeita solicitação	PUT /api/access/:id/reject, JWT admin	200 OK: status='rejected'	PASSOU
CT-026	Controle de Acesso	Contagem de pendentes para badge do admin	GET /api/access/pending/count, JWT admin	200 OK: { count: N }	PASSOU
CT-027	Exportação CSV	Download de CSV da última semana	Clique no botão 'Exportar CSV' no dashboard com dados existentes	Arquivo .csv com BOM UTF-8, separador ','	PASSOU
CT-028	Exportação CSV	CSV vazio quando não há dados na semana	Botão 'Exportar' sem registros na última semana	Arquivo CSV gerado apenas com cabeçalho; sem erro	PASSOU

SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

ID	Módulo	Cenário	Entrada	Saída Esperada	Resultado
CT-029	Segurança	Headers HTTP de segurança (Helmet)	Qualquer requisição GET ao backend	Headers X-Content-Type-Options, X-Frame-Options etc.	PASSOU
CT-030	Segurança	CORS bloqueia origem não autorizada	Requisição cross-origin de domínio não listado em FRONTEND_ORIGIN	Erro CORS; acesso negado pelo middleware	PASSOU

6.3 Validação com Usuários

Durante os testes como usuários finais, foi possível simular processos do dia a dia de um responsável pelo sistema. A maioria das funcionalidades atendeu ao esperado, mas alguns pontos que poderiam causar dificuldades em situações reais foram identificados:

- Os administradores não conseguiam promover outros usuários a administradores.
- Não havia como remover usuários do domínio.
- Caso um administrador recusasse um acesso e depois o concedesse manualmente, o usuário não conseguia visualizar os dispositivos corretamente.
- O usuário comum não tinha autonomia para sair de um dispositivo ao qual tinha acesso.
- Ausência de opções de filtros entre os widgets do dashboard.

Após essas identificações, as dificuldades foram registradas. As de origem lógica foram corrigidas imediatamente no código-fonte; as que demandavam novas funcionalidades foram modeladas e registradas no backlog para implementação planejada.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

7. IMPLANTAÇÃO E MANUTENÇÃO

7.1 Plano de Implantação

O sistema chega ao cliente final através da hospedagem em nuvem (Vercel e Render). Basta o cliente acessar <https://cleanairsenailauro.vercel.app/>, realizar o cadastro e configurar seu ambiente de monitoramento.

O processo de CI/CD é executado automaticamente no serviço de hospedagem configurado com as pipelines, via Github Actions, assim: tanto o Render (backend) quanto a Vercel (frontend) estão configurados para detectar novos commits / merges na *main* (na *cloud-database* somente pull requests e commits) do repositório GitHub e iniciar o build e o deploy automaticamente, sem intervenção manual em caso de aprovação dos processos.

7.2 Documentação Técnica

O repositório conta com uma pasta chamada “/documentacoes” em que estão presentes as principais documentações ligadas ao sistema para melhor entendimento. Já a documentação formal da API REST (Swagger/OpenAPI) está acessível pelo endpoint */api-docs* do backend em produção. A especificação foi construída com a biblioteca *swagger-jsdoc*, integrada aos schemas de validação Zod já utilizados pelo sistema, o que garante que as regras de validação documentadas reflitam exatamente as regras aplicadas em tempo de execução.

7.3 Manutenção e Suporte

Como todo sistema, podem surgir problemas após a entrega ao público. O Clean Air disponibilizará um canal de suporte por e-mail: contato.cleanair.suporte@gmail.com, pelo qual os usuários poderão reportar dificuldades, contribuindo para a identificação e o planejamento de correções.

Adicionalmente, a manutenção será monitorada pelos recursos de logs dos serviços de hospedagem (Render Logs para o backend e Neon Console para o banco de dados), permitindo analisar causas de falhas com maior controle e facilidade.

8. MODELO DE NEGÓCIOS

A partir desta seção, o documento integra o Plano de Negócios do Clean Air, abordando o mercado, a proposta de valor, o modelo de receita, as estratégias comerciais, o plano financeiro e os indicadores de sucesso.

8.1 Análise de Mercado

8.1.1 Segmentação de Mercado

Foram identificados os seguintes segmentos-alvo, com seus respectivos tamanhos estimados e disposição a pagar:

Segmento	Tamanho Estimado	Disposição a Pagar	Prioridade
Pequenos comerciantes com refrigeração (açougues, padarias, sorveterias, farmácias)	520.000 estabelecimentos	R\$ 35–75/mês	Prioritário
Pequenos produtores rurais com estufas e ambientes sensíveis	180.000 produtores	R\$ 35–75/mês	Prioritário
Entusiastas de IoT, técnicos autônomos e makers	60.000 pessoas	R\$ 0–35/mês	Secundário

8.1.2 Vantagens Competitivas e Desvantagens a Mitigar

Vantagens competitivas: o Clean Air concentra-se em entregar somente os recursos mais essenciais e abrangentes para o monitoramento IoT (gerência simplificada e funcionalidades abstraídas), resultando em uma interface mais limpa e atrativa para usuários menos técnicos, com configuração rápida e sem necessidade de download de recursos externos. A comparação detalhada com os concorrentes consta na Seção 4.1.

Desvantagens a mitigar: equipe reduzida em relação aos concorrentes, disponibilidade financeira limitada, ausência de histórico de mercado e recursos de marketing restritos. As estratégias de mitigação incluem parcerias institucionais com programas de apoio a projetos em ascensão, como o Sebrae e o Pequenas Empresas & Grandes Negócios, que auxiliariam na divulgação.

8.1.3 Análise SWOT

Forças (Strengths)	Fraquezas (Weaknesses)
Conhecimento técnico da equipe; baixo custo de desenvolvimento; solução específica para um problema real; flexibilidade para customização.	Equipe reduzida; recursos financeiros limitados; ausência de histórico no mercado; dependência de tecnologias específicas.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

Oportunidades (Opportunities)	Ameaças (Threats)
Crescimento do setor de tecnologia; demanda por soluções de baixo custo; incentivos à inovação; parcerias com instituições.	Concorrentes estabelecidos; mudanças tecnológicas rápidas; crise econômica que reduza o orçamento dos clientes; questões de segurança de dados e regulamentação; descontinuidade do protocolo MQTT.

8.2 Proposta de Valor

O Clean Air resolve o problema identificado - A dificuldade de encontrar plataformas com configuração simples de dispositivos MQTT- De forma eficiente e acessível. Os benefícios para os que vierem a utilizar incluem: economia de tempo com deslocamento, redução de custos, auxílio na tomada de decisão e controle de acesso.

O diferencial em relação aos concorrentes está na combinação de preço acessível, funcionalidades abstraídas, interface minimalista e curto processo de configuração. Enquanto os concorrentes oferecem soluções abrangentes, com preços mais elevados e configuração longa, o Clean Air propõe um produto focado, de custo reduzido e adaptado às necessidades mais recorrentes dos usuários.

8.3 Modelo de Receita

O modelo de receita adotado é o SaaS (Software as a Service) com planos mensais, somado a uma estratégia Freemium (versão gratuita limitada). Os planos são:

Plano	Usuários	Dispositivos	Recurso Adicional	Preço/mês	Público
Gratuito	2	3	Funcionalidades básicas (verificação)	R\$ 0,00	Usuários com poucos ambientes e entusiastas
Comercial	5	10	Tabela de exceções em todos os dispositivos	R\$ 35,00	MEIs e pequenos comerciantes com espaço reduzido
Empresarial	25	50	Alertas via Telegram	R\$ 75,00	Pequenas empresas e agricultores de pequeno/médio porte

Nota de padronização: trechos da versão original do Plano de Negócios referiam-se a um "plano Pro". Esse termo foi unificado para os planos Comercial e Empresarial, eliminando ambiguidade.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

8.3.1 Projeção de Receita (run-rate)

As projeções abaixo representam a receita em regime (run-rate), ou seja, a receita mensal de saída de cada ano multiplicada por 12. A composição da base pagante assume 70% no plano Comercial e 30% no Empresarial, resultando em ticket médio de R\$ 47,00/assinante.

Ano	Usuários Gratuitos	Usuários Pagantes	Receita Mensal (R\$)	Receita Anual (R\$)
Ano 1	200	50	2.350,00	28.200,00
Ano 2	1.000	200	9.400,00	112.800,00
Ano 3	2.500	500	23.500,00	282.000,00

Observação de realismo: por se tratar de receita em regime, a receita efetiva do Ano 1 é menor, pois a base pagante cresce ao longo dos meses (ver Seção 8.7.3). Considerando a curva de adoção, a receita efetiva do primeiro ano situa-se em torno de R\$ 12.000,00 a R\$ 14.000,00.

8.4 Canais de Distribuição e Estratégia de Marketing

O Clean Air chegará ao cliente por meio de dois canais principais: o canal direto (site oficial, para cadastro e assinatura) e as redes sociais (divulgação orgânica e anúncios segmentados para alcançar o público correto).

A estratégia de marketing está voltada para a produção de conteúdo demonstrativo do uso da solução, participação em eventos de inovação (como os do Sebrae), inscrição no programa Pequenas Empresas & Grandes Negócios (PEGN) e um programa de indicação com descontos para usuários que atraírem novos clientes.

Custo de Aquisição de Cliente (CAC): estima-se um CAC inicial de R\$ 1.800,00 por cliente no canal digital pago, com redução progressiva à medida que o reconhecimento da marca cresce. Dado o orçamento de marketing limitado, a estratégia prioriza o crescimento orgânico e o programa de indicação, perseguindo a meta de CAC blended de até R\$ 50,00 por cliente definida nos indicadores (Seção 8.7). A maior parte das aquisições, portanto, deve vir de canais orgânicos e de indicação, e não de tráfego pago.

8.5 Parcerias Estratégicas

Parceiros relevantes para o sucesso do projeto incluem:

- Provedor de infraestrutura em nuvem.
- Gateway de pagamento (PagSeguro, Mercado Pago).
- APIs de serviços complementares (notificações via Telegram).
- Instituições parceiras para testes e validação.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

As integrações com a API de mensagens do Telegram serão desenvolvidas seguindo as melhores práticas de consumo de APIs REST, para fornecer as funcionalidades de alerta previstas no plano Empresarial.

8.6 Plano Financeiro

Esta seção apresenta um modelo financeiro coerente, separando duas perspectivas que antes estavam misturadas: (a) a **viabilidade da empresa**, baseada na receita real de assinaturas; e (b) o **retorno da solução para o cliente** (visão de valor), baseado na planilha de ROI elaborada. Os indicadores agregados anteriores (ROI de 185%, VPL e TIR) foram revisados por não possuírem memória de cálculo de suporte, sendo substituídos por indicadores transparentes e rastreáveis.

8.6.1 Investimento Inicial (CAPEX)

O investimento inicial inclui o custo de desenvolvimento, contabilizado como investimento real conforme as horas dedicadas pela equipe:

Item	Descrição	Valor (R\$)
Desenvolvimento	4 desenvolvedores Jr., ~150 horas, a R\$ 35,00/hora	21.000,00
Treinamento	Treinamento para uso do sistema	350,00
Hardware (piloto)	Frota de demonstração: conjuntos ESP32 + sensores (ex.: kit DHT22)	1.200,00
Marketing e vendas (1º ano)	Anúncios, materiais de divulgação e deslocamento para eventos	5.000,00
Serviços de terceiros (1º ano)	E-mail, monitoramento e demais serviços	1.500,00
Subtotal		29.050,00
Contingência (30%)	Reserva sobre itens variáveis (hardware, marketing e serviços)	2.310,00
Investimento Total		31.360,00

O desenvolvimento responde por cerca de 67% do investimento. Por se tratar de um projeto de TCC, esse custo é, na prática, não desembolsado — porém é mantido no cálculo para refletir o custo real de reprodução da solução em um cenário comercial.

8.6.2 Estrutura de Custos Operacionais (OPEX)

Os custos recorrentes de operação, já corrigidos os erros aritméticos da versão anterior, são:

Tipo	Item	Valor Mensal (R\$)	Valor Anual (R\$)
Fixo	Hospedagem em nuvem (Vercel, Render e Neon)	644,00	7.728,00



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

Tipo	Item	Valor Mensal (R\$)	Valor Anual (R\$)
Fixo	Serviços SaaS de terceiros (e-mail com domínio)	80,00	960,00
Variável	Marketing digital contínuo	600,00	7.200,00
Variável	Taxas de pagamento/gateway (~3,5% s/ receita)	82,25	987,00
Total		1.406,25	16.875,00

Observa-se que o marketing aqui é o esforço contínuo (R\$ 600,00/mês); o investimento de lançamento de R\$ 5.000,00 está contabilizado no CAPEX (Seção 8.6.1).

8.6.3 Cenários de Projeção de Receita

Cenário	Premissa	Ano 1 (R\$)	Ano 2 (R\$)	Ano 3 (R\$)
Conservador	30% das metas de pagantes; adoção mais lenta	8.460,00	33.840,00	84.600,00
Realista	100% das metas; adoção gradual consistente	28.200,00	112.800,00	282.000,00
Otimista	150% das metas; adoção rápida e viralização	42.300,00	169.200,00	423.000,00

Os valores são run-rate (regime). Reitera-se que, com a curva de adoção, a receita efetiva do Ano 1 é inferior à de regime (ver Seção 8.3.1).

8.6.4 Viabilidade Financeira da Empresa (visão de caixa por assinaturas)

Indicador	Valor	Interpretação
Break-even de infraestrutura	~19 assinantes Comerciais	$19 \times R\$ 35,00 = R\$ 665,00$ cobrem o custo fixo de hospedagem (R\$ 644,00/mês).
Break-even operacional total	~40 assinantes Comerciais	Cobre o OPEX total (R\$ 1.406,25/mês), patamar próximo da meta de 50 pagantes ao fim do Ano 1.
Margem operacional (regime, Ano 1)	~40%	Receita de regime (R\$ 28.200) menos OPEX (R\$ 16.875) $\approx$ R\$ 11.325, ou ~40%. A margem cresce nos anos seguintes pela diluição dos custos fixos.
Payback do investimento	~18 a 24 meses	No cenário realista, a margem operacional acumulada recupera o investimento total (R\$ 31.360) ao longo do 2º ano de operação.

Leitura de realismo: a estrutura do negócio é sólida — modelo SaaS Freemium, três planos e custo marginal de infraestrutura baixo. As premissas mais sensíveis (e otimistas) são a taxa de conversão de gratuitos em pagantes e o ritmo de aquisição frente ao orçamento de marketing. Recomenda-se trabalhar com conversão de 5% a



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

10% no primeiro ano (em vez dos ~20% implícitos na projeção original) e revisar os indicadores conforme dados reais de operação.

8.6.5 Retorno da Solução para o Cliente (visão de valor)

A planilha de ROI elaborada demonstra o valor que a solução gera, justificando a disposição a pagar. Ela confronta o custo total de implantação da solução (R\$ 22.299,00, incluindo desenvolvimento, treinamento, hardware e a primeira mensalidade de hospedagem) com o valor mensal mensurável gerado (R\$ 4.050,00), composto por economia de tempo (R\$ 600), controle de dados (R\$ 1.200), histórico prolongado (R\$ 800), alertas de exceções (R\$ 150), economia em planos subutilizados (R\$ 800) e gestão de acessos (R\$ 500).

Importante: o valor de R\$ 4.050,00/mês representa o benefício percebido pela solução, e não a receita de assinaturas da empresa (tratada na Seção 8.6.4). A evolução do retorno é:

Mês	Valor Investido (R\$)	Valor Gerado Acum. (R\$)	ROI
Mês 1	22.299	4.050	-81,8%
Mês 3	23.587	12.150	-48,5%
Mês 6	25.519	24.300	-4,8%
Mês 7	26.163	28.350	+8,4%
Mês 9	27.451	36.450	+32,8%
Mês 12	29.383	48.600	+65,4%

O retorno da solução torna-se positivo a partir do 7º mês, atingindo aproximadamente +65% ao final de 12 meses — um forte argumento de valor para o cliente, dado que ele paga R\$ 35,00 a R\$ 75,00/mês por uma solução que lhe gera benefícios mensuráveis muito superiores.

8.7 Indicadores de Sucesso

8.7.1 KPIs Técnicos

Indicador	Meta	Frequência
Uptime do sistema (Render/Vercel)	≥ 99,5%	Mensal
Tempo médio de resposta da API REST	< 1,5 s	Semanal
Taxa de erro (payloads malformados bloqueados)	< 1%	Diário
Tempo de recuperação (RTO) após falhas	< 4 horas	Por incidente



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

8.7.2 KPIs de Negócio

Indicador	Meta (Ano 1)	Frequência
Usuários cadastrados	500	Mensal
Usuários ativos no mês (MAU)	200	Mensal
Taxa de conversão (gratuito → pago)	5% a 10%	Mensal
CAC (blended)	≤ R\$ 50,00	Mensal
LTV (Lifetime Value)	≥ R\$ 300,00	Trimestral
Churn rate (plano pago)	< 5% ao mês	Mensal
NPS (Net Promoter Score)	≥ 60	Trimestral

8.7.3 Metas de Crescimento

Período	Usuários Cadastrados	Usuários Pagantes	Receita Mensal (R\$)
Mês 1	5	0	0,00
Mês 3	30	5	215,00
Mês 6	100	20	820,00
Mês 12	200	50	2.350,00

8.8 Sustentabilidade e Crescimento

8.8.1 Roadmap de Funcionalidades

Versão	Período	Funcionalidades	Objetivo de Negócio
v1.0 (MVP)	Dez/2025	Funcionalidades essenciais para demonstrar a solução	Validar a proposta de valor
v1.1	Abr/2026	Melhorias visuais e informativas	Aumentar a satisfação e a comunicação com o usuário
v2.0	Abr/2026	Divisão por domínios e controle de acessos	Garantir o funcionamento da arquitetura multitenant
v3.0	Mai/2026	Implementação de scripts de CI/CD	Práticas profissionais para melhorias futuras seguras

8.8.2 Plano de Escalabilidade

A arquitetura do Clean Air foi projetada para escalar horizontalmente com baixo custo marginal. O backend stateless no Render e o banco serverless no Neon permitem crescimento sem grandes saltos de investimento em infraestrutura, com custos estimados entre 12% e 20% da receita operacional:

- **Até 50 usuários:** planos básicos de Vercel (Pro), Render (Starter) e Neon (Launch), além de domínio e e-mail — custo estimado de R\$ 250,00 a R\$ 400,00/mês.



SENAI REGIONAL METROPOLITANA

Lauro de Freitas/Bahia

- **Até 500 usuários:** upgrade dos serviços de hospedagem e banco para suportar maior volume de acessos, conexões WebSocket e armazenamento de histórico — custo estimado de R\$ 600,00 a R\$ 1.200,00/mês.
- **Até 5.000 usuários:** infraestrutura mais robusta, com múltiplas instâncias de backend, otimizações de banco e monitoramento avançado — custo estimado de R\$ 2.000,00 a R\$ 5.000,00/mês.
- **Acima de 50.000 usuários:** arquitetura distribuída com balanceamento de carga e armazenamento escalável — custos superiores a R\$ 8.000,00/mês.

8.8.3 Estratégia de Retenção de Clientes

Programa de fidelização: clientes com assinatura ativa por 12 meses consecutivos recebem 1 mês gratuito; usuários que indicarem novos assinantes podem receber descontos na mensalidade seguinte, incentivando o crescimento orgânico e reduzindo o CAC.

Ciclo de melhoria contínua: novas funcionalidades serão lançadas periodicamente com base no feedback dos usuários, coletado por formulários e pesquisas de satisfação, e comunicadas por e-mail e notificações no sistema.

Suporte ao cliente: clientes dos planos pagos terão suporte por e-mail, com acompanhamento periódico da satisfação via NPS (meta mínima de 60 pontos ao fim do primeiro ano).

9. RISCOS E DESAFIOS

Os riscos a seguir consolidam, sem duplicidade, os riscos técnicos, de negócio e operacionais identificados ao longo do projeto.

9.1 Riscos Técnicos

Risco	Prob.	Impacto	Estratégia de Mitigação
Sobrecarga do banco com grandes volumes na tabela mqtt_data	Média	Alto	Paginação nas consultas e política de retenção de dados (backlog B009); monitorar o crescimento via Neon Console.
Degradação dos gráficos com volume excessivo de pontos	Alta	Médio	Filtros de período customizáveis nos widgets (backlog B004); limitar a exibição aos últimos N pontos.
Breaking changes em bibliotecas (Chart.js, mqtt, Socket.IO)	Baixa	Alto	Manter versões fixadas no package.json; avaliar migração para Apache ECharts
Exposição do tópico MQTT público (broker.hivemq.com)	Média	Alto	Recomendar broker privado com autenticação; alertar no cadastro de dispositivo sobre o risco de brokers públicos.
Instabilidade/queda de Wi-Fi no ESP32	Média	Alto	Rotinas automáticas de reconexão (auto-reconnect) no firmware em C/C++ do microcontrolador, evitando o travamento da placa.
Performance/latência sob carga de payloads MQTT	Baixa	Alto	Otimizar consultas assíncronas no Node.js; usar connection pooling estável com o Neon; testes com múltiplos sensores concorrentes antes do lançamento.
Vulnerabilidade de segurança em dados ou rotas da API	Média	Crítico	Hashing de senhas (bcrypt), validação estrita de inputs via Zod e proteção de rotas via JWT; headers de segurança via Helmet.

9.2 Riscos de Negócio

Risco	Prob.	Impacto	Estratégia de Mitigação
Baixa adoção da plataforma por receio de tecnologia	Média	Alto	Anúncios que atestem a praticidade da solução, com relatos de usuários reais, para tranquilizar novos clientes.
Concorrentes (Blynk/Thingsboard) lancem versões em português	Alta	Médio	Foco no nicho desatendido (pequenas estufas e segurança ocupacional) e na agilidade do suporte local personalizado.
Dificuldade de monetização no plano Empresarial	Média	Médio	Validar a retenção via plano gratuito (Freemium) e coletar feedback de usabilidade antes de aplicar travas rígidas de cobrança.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

9.3 Riscos Operacionais

Risco	Prob.	Impacto	Estratégia de Mitigação
Vazamento de dados sensíveis ou credenciais de usuários	Baixa	Crítico	Criptografia em trânsito (HTTPS/SSL nativos da Vercel e Render) e políticas de privilégio mínimo no banco Neon.
Prazo insuficiente para conclusão e refinamento de telas	Média	Alto	Priorização rigorosa das funcionalidades do MVP (cartões em tempo real e tabela de excedências).
Atraso por indisponibilidade da equipe na conclusão do TCC	Média	Médio	Sprints com escopo definido, gestão via Trello e priorização rigorosa do backlog.
Descontinuidade/mudança nos planos gratuitos de hospedagem (Render, Vercel, Neon)	Baixa	Alto	Manter o Plano de Implantação atualizado para migração rápida a serviços alternativos (Railway, Supabase, Netlify).

10. CONCLUSÃO E PRÓXIMAS ETAPAS

10.1 Considerações Finais

O Clean Air, após o período de desenvolvimento, alcançou os objetivos propostos inicialmente: capta dados MQTT em formato de payload nos tópicos informados, possui uma interface minimalista e usual seguindo os princípios de Nielsen, conta com as funcionalidades necessárias para boa parte do público — testadas e operantes — e está estruturado de forma modular, podendo receber novas funcionalidades sem quebrar a aplicação.

Quanto ao aprendizado técnico da equipe, o processo foi extremamente frutuoso: versionamento com boas práticas, estratégias de branches, processos de engenharia de software, etapas de planejamento e documentação, execução e realização de testes, definição e aplicação de CI/CD, e processos de manutenção e implantação de sistemas. Além disso, desenvolveram-se competências de trabalho em equipe, organização, planejamento e tomada de decisão.

10.2 Resultados Esperados

Espera-se que a solução traga um sentimento de inclusão tecnológica aos usuários do Clean Air, permitindo que pessoas com menor afinidade com tecnologia obtenham, para seus negócios, melhor visão macro, controle de segurança e gestão de dados — participando do processo de difusão da Indústria 4.0.

10.3 Viabilidade Geral do Projeto

Com base na análise de mercado, na estrutura de custos e nas projeções financeiras, o projeto Clean Air é considerado viável sob os seguintes aspectos:

- **Viabilidade técnica:** a equipe possui as competências necessárias e as tecnologias selecionadas são adequadas ao escopo.
- **Viabilidade econômica:** o break-even de infraestrutura ocorre com ~19 assinantes Comerciais e o break-even operacional total próximo de 40 assinantes; o payback do investimento estimado situa-se entre 18 e 24 meses no cenário realista.
- **Viabilidade comercial:** existe demanda real, os concorrentes não atendem completamente o nicho e a proposta de valor é clara e diferenciada (idioma português, simplicidade e configuração rápida).
- **Viabilidade operacional:** a equipe está dimensionada para o escopo e o cronograma é factível dentro do prazo disponível.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

10.4 Próximas Etapas

Por motivos da evolução de uma solução como essa, nem sempre é possível trazer tudo que está proposto de uma só vez, entretanto, durante o desenvolvimento e sua conclusão, entre as ideias que podem vir a serem implementadas futuramente para um refinamento e agregação de valor do Clean Air estão as seguintes:

- Envio de ações pela solução para os microcontroladores - Pelo clean ser possível o usuário enviar comandos pré configurados que serão realizados pelo microcontrolador que está à distância
- Integração com câmeras de Segurança - YOLO. O sistema poderá incorporar um módulo de integração com câmeras de segurança para monitoramento em tempo real. Utilizando modelos de visão computacional baseados em YOLO, seria possível detectar e acompanhar a movimentação de pessoas em áreas monitoradas.
- Autenticação via Contas Google: Seria a capacidade de os usuários poderem realizar seu cadastro através diretamente de suas contas google, agilizando o processo e trazendo uma maior confiabilidade para novos utilizadores.
- Serviço de marketplace para configuração e venda de microcontroladores: A existência de uma outra página que poderia direcionar para o Clean air, esse serviço serviria de alternativa para realização do cadastro e compra dos planos, como que também, nele seria possível realizar a compra de Esp's 32, sensores e serviços de configuração dos dispositivos.



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

REFERÊNCIAS

ABES — Associação Brasileira das Empresas de Software. Mercado Brasileiro de Software e Serviços 2025. São Paulo: ABES, 2025.

EXPRESS. Express.js — Fast, unopinionated, minimalist web framework for Node.js. Disponível em: <https://expressjs.com/>. Acesso em: jun. 2026.

HIVEMQ. HiveMQ — MQTT Platform for the IoT. Disponível em: <https://www.hivemq.com/>. Acesso em: jun. 2026.

INSTITUTO ATLÂNTICO. Quais as barreiras na adoção da IoT na indústria e como o Atlântico e a Embrapii podem ajudar. 2024. Disponível em: <https://www.atlantico.com.br/blog/quais-as-barreiras-na-adocao-da-iot-na-industria-e-como-o-atlantico-e-a-embrapii-podem-ajudar/>. Acesso em: jun. 2026.

MQTT.JS. MQTT.js — a client library for the MQTT protocol. Disponível em: <https://github.com/mqttjs/MQTT.js>. Acesso em: jun. 2026.

NEON. Neon Documentation — Serverless PostgreSQL. Disponível em: <https://neon.tech/docs/>. Acesso em: jun. 2026.

NIELSEN, Jakob. 10 Usability Heuristics for User Interface Design. Nielsen Norman Group, 1994. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: jun. 2026.

NODE.JS. Node.js Documentation. Disponível em: <https://nodejs.org/docs/>. Acesso em: jun. 2026.

PRESSMAN, Roger S. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: Bookman, 2016.

REACT. React Documentation. Disponível em: <https://react.dev/>. Acesso em: jun. 2026.

SEBRAE. Maturidade Digital dos Pequenos Negócios no Brasil. Brasília: Sebrae, 2024. Disponível em: <https://sebraepr.com.br/impulsiona/maturidade-digital-dos-pequenos-negocios-no-brasil-2024/>. Acesso em: jun. 2026.

SOCKET.IO. Socket.IO Documentation. Disponível em: <https://socket.io/docs/>. Acesso em: jun. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

TONET, Ivan. Milhões de PMEs e um desafio: a transformação digital dos negócios. Inovativos, 9 out. 2024. Entrevista. Disponível em: <https://inovativos.com.br/2024/10>



SENAI REGIONAL METROPOLITANA
Lauro de Freitas/Bahia

[/09/milhoes-de-pmes-e-um-desafio-a-transformacao-digital-dos-negocios/](#). Acesso em: jun. 2026.

ZOD. Zod — TypeScript-first schema validation. Disponível em: <https://zod.dev/>. Acesso em: jun. 2026.